WEEK 07

Exceptions and with-blocks

This week: learn, practise, build

01 · Python

Exceptions and with-blocks
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

All-or-nothing reservation
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Exceptions and with-blocks
Concept → trace a small example → explain its behavior

Exceptions, assignments and transaction boundaries

Return or raise

A return supplies a result; an exception interrupts the normal path.

Assignment order

The right side must finish before the new value is assigned.

Context behavior

A SQLite transaction can roll back database writes, not arbitrary Python changes.

Check which statements ran and which stored values changed.

Exceptions interrupt normal execution

Signal a failure

raise stops the current path. A function does not return its normal result.

Handle at a boundary

except handles a matching failure. Assignment waits until the right side returns successfully.

Use a managed block

with asks a context manager to enter and exit a block. Cleanup behavior depends on that manager.

Understand the behavior before changing the example.

Check the outcome and the state

1

Arrange

Known starting database
Independent expectation

2

Act

Success or forced failure
Let exceptions cross the boundary

3

Inspect

Check result or error
Compare every affected table

Read the diagram, then trace the same steps in code.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
raise ValueError(...)	Stop the current path	Signal a known invalid value
try / except	Choose an error path	Handle a specific exception
with db:	A transaction boundary	Observe normal or failed exit
assertRaises(...)	Check an expected error	Also inspect the resulting state

Python Basics · sections 09, 13, 17 · use only when you need a lookup.

Check exact type before a numeric range

```
01 value = True
02 print(type(value) is int)
03 value = "2"
04 print(type(value) is int and value > 0)
05 value = 2
06 print(type(value) is int and value > 0)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

An exception does not undo ordinary mutation

```
state = {"count": 5}
try:
    state["count"] -= 1
    raise ValueError("later step failed")
except ValueError:
    print(state["count"])
assert state["count"] == 4
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

A failed call leaves its assignment unfinished

```
01 def checked_count(value):
02     if value < 0:
03         raise ValueError("negative count")
04     return value
05 count = 5
06 try:
07     count = checked_count(-1)
08 except ValueError:
09     print("rejected")
10 print(count)
```

1. Expected: rejected, then 5.
2. The right side raises before count is reassigned.
3. Change -1 to 2 and predict the new output.

Predict → execute → explain the value at each step.

assertRaises: check the exception and state

```
01  import unittest
02
03  def reserve_count(available, requested):
04      if requested > available:
05          raise ValueError("not enough containers")
06      return available - requested
07
08  check = unittest.TestCase()
09  available = 3
10  with check.assertRaises(ValueError):
11      available = reserve_count(available, 4)
12  check.assertEqual(available, 3)
```

1. assertRaises checks the exception type.
2. Failed assignment keeps the old value.
3. Database rollback needs its own state check.

Check the expected exception and the state left afterward.

Hands-on practice

Week_07.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1–P2 · Easy

Use the approach hints. Write the code yourself in a blank cell.

P3 · Core build

Implement a complete small program. Write expected results and assertions.

P4–P5 · Optional

Apply the skill to a new case, then attempt a harder challenge if time permits.

Write your own code, predict the result, then test and explain it.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Trace raise, except and a with-block boundary; inspect the remaining state.

Project studio

All-or-nothing reservation
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

A booking must not leave only part of a request set aside.

2

This week, you will

Accept the whole booking, or leave the records unchanged if it cannot finish.

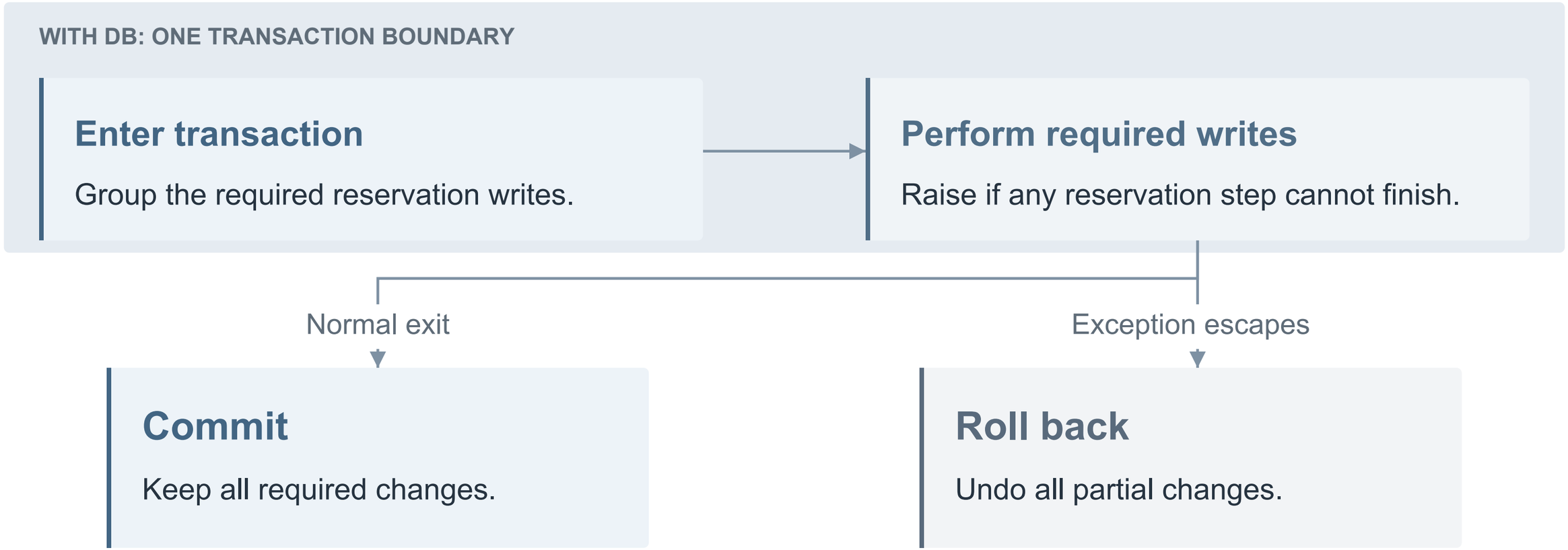
3

A useful result

Another volunteer sees consistent available counts.

Use the supplied starting files. Focus on one useful result and check it.

Let a required failure escape before handling it



Catch the failure after the transaction has had a chance to roll back.

Reference: Observe rollback after a deliberate interruption

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE availability(containers INT)")
with db: db.execute("INSERT INTO availability VALUES (5)")
try:
    with db:
        db.execute("UPDATE availability SET containers=3")
        raise RuntimeError("practice interruption")
except RuntimeError: pass
row = db.execute("SELECT * FROM availability").fetchone()
assert row == (5,)
db.close()
```

1. Commit the opening state first.
2. Let failure escape the with block.
3. Read the state after handling the error.

An escaping exception rolls back writes in the transaction.

Conditional UPDATE with placeholders

```
UPDATE offer_batches
SET available_containers = available_containers - ?
WHERE batch_id = ?
  AND available_containers >= ?;
```

1. Bind requested count, batch ID, requested count
2. Inspect cursor.rowcount after execute
3. Zero does not automatically raise a failure

A zero-row update must be inspected and handled explicitly.

rowcount: one row changed

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE offers(id TEXT, available INT)")
db.execute("INSERT INTO offers VALUES ('DEMO',5)")
cur = db.execute(
    "UPDATE offers SET available=available-? "
    "WHERE id=? AND available>=?", (2, "DEMO", 2))
assert cur.rowcount == 1
row = db.execute("SELECT * FROM offers").fetchone()
assert row[-1] == 3
db.close()
```

1. Inspect the returned cursor.
2. rowcount is 1 after a match.
3. Query again to observe the value.

A matching conditional UPDATE changes one row.

rowcount: no row matched

```
import sqlite3
db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE offers(available INT)")
db.execute("INSERT INTO offers VALUES (3)")
cur = db.execute(
    "UPDATE offers SET available=available-? "
    "WHERE available>=?", (4, 4))
assert cur.rowcount == 0
row = db.execute("SELECT * FROM offers").fetchone()
assert row[-1] == 3
db.close()
```

1. rowcount is 0 when the condition fails.
2. The stored value stays the same.
3. The caller decides whether to raise.

No matching row means no update; it does not raise automatically.

Your implementation TODO

Supplied starting point

Database fixture starts Q11=4, Q12=4; schema, starter and failure-injection hook are supplied.

You implement / do

Validate review, counts and batches; apply guarded updates in one transaction. Raise on shortage; let failure escape the with-block before handling it.

Check the evidence

Unreviewed, shortage and injected failure must leave state unchanged. Success Q11:2 + Q12:1 → remaining 2,3; one header and two lines.

Also check zero counts, unknown batches and repeated batch lines.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Implement reserve in one transaction; reject bad requests.
3 · Check	Success leaves 2/3; refusals leave every table unchanged.
4 · Explain	Show your code, its actual result and why it behaves that way.

Also check zero counts, unknown batches and repeated batch lines.

Further reading in English

1. ## SQLite transactions

   https://www.sqlite.org/lang_transaction.html

2. ## Python sqlite3 connection context manager

   https://docs.python.org/3/library/sqlite3.html#how-to-use-the-connection-context-manager

Read one relevant section, then bring an example and a question.